

文献研究与思路总结

2023-NIPS-Prediction and Control in Continual Reinforcement Learning

2017-Neural Episodic Control

2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models



问题介绍

持续强化学习：是强化学习（RL）与持续学习（CL）的交叉领域，旨在让智能体在**非静态环境**中通过与环境的持续交互，不断积累新知识、适应新任务，同时避免遗忘旧知识。现存的两大问题：

①**灾难性遗忘**：灾难性遗忘是指智能体在学习新任务（或适应环境变化）的过程中，其神经网络参数发生剧烈更新，导致先前学到的旧任务知识（策略、价值函数或特征表示）的现象。

②**可塑性丧失**：可塑性丧失是指随着智能体经历的任务序列越来越长，或者训练时间越来越久，神经网络逐渐失去学习新知识的能力。即使面对全新的、简单的任务，智能体也无法像初始状态那样快速收敛或达到高性能
也称为**稳定性与可塑性难题**：当环境发生变化时，RL为了学习新估计而遗忘过去的预测，还是保留旧估计（它们未来可能再次有用），从而牺牲在新任务上的准确性

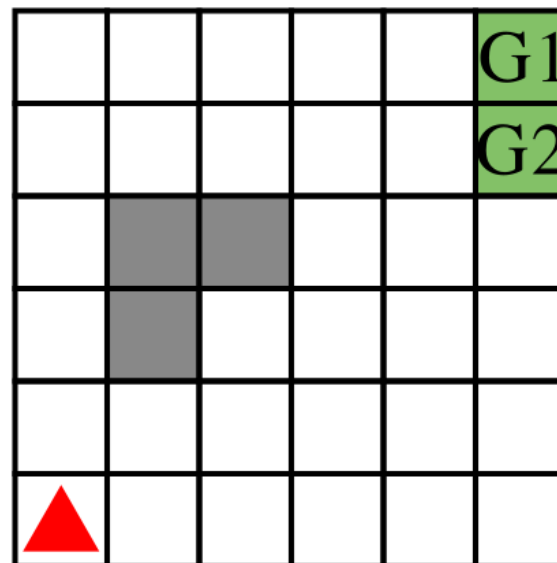
例如：

任务1：到G1加1分，到G2减1分，RL经过反复试错学会了最优路径，并建立了价值函数： $V(G1)=1$ ， $V(G2)=-1$ （旧知识，已经稳定收敛）

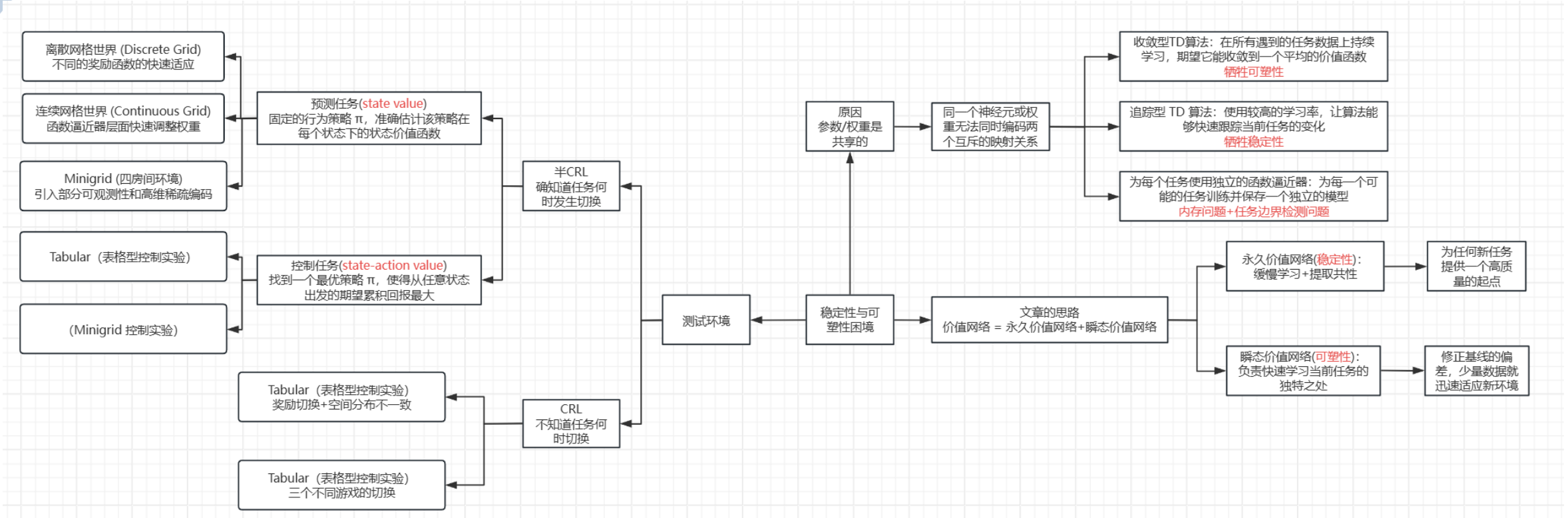
任务2：到G1减1分，到G2加1分： $V(G1)=-1$ ； $V(G2)=1$

①当任务2开始训练时，若沿用旧的值函数更新：降低 $V(G1)$ ，提高 $V(G2)$ ，假设再遇到任务1，那模型会认为 $V(G1)$ 的价值低，需要重新花大量时间学习才能恢复对任务1的重新学习（**灾难性遗忘**）

②冻结旧的值函数：任务1训练好以后冻结参数，这样的话无论何时遇到任务1都会效果很好，但是遇到任务2效果就会很差（**可塑性丧失**）



相关研究：2023-NIPS-Prediction and Control in Continual Reinforcement Learning



文中的思路:

永久网络: 通过定期从瞬态网络中蒸馏经过验证的优质策略来低频更新, 将瞬态网络学到的特定任务经验转化为跨任务通用的底层规律和稳健技能, 在确保持续积累新知识的同时, 有效防止对旧技能的灾难性遗忘。

瞬态网络: 直接与环境交互并利用最新的实时数据高频更新, 专门负责捕捉当前任务特有的动态变化或短期策略; 其核心优势在于极高的可塑性, 能够迅速收敛以适应新环境。

算法流程：Prediction and Control in Continual Reinforcement Learning

半CRL测试环境

Algorithm 1 PT-TD learning (Prediction)

```
1: Initialize: Buffer  $\mathcal{B}$ ,  $\theta$ ,  $\mathbf{w}$ 
2: for  $t : 0 \rightarrow \infty$  do
3:   Store  $S_t$  in  $\mathcal{B}$ 
4:   Observe  $R_{t+1}, S_{t+1}$ 
5:   Update  $\mathbf{w}$  using Eq. (5)
6:   if Task Ends then
7:     Update  $\theta$  using  $\mathcal{B}$  and Eq. (4)
8:     Reset transient value function,  $\mathbf{w}$ 
9:     Reset  $\mathcal{B}$ 
10:  end if
11: end for
```

Algorithm 3 PT-Q-learning (Control)

```
1: Initialize: Buffer  $\mathcal{B}$ ,  $\theta$ ,  $\mathbf{w}$ 
2: for  $t : 0 \rightarrow \infty$  do
3:   Take action  $A_t$ 
4:   Store  $S_t, A_t$  in  $\mathcal{B}$ 
5:   Observe  $R_{t+1}, S_{t+1}$ 
6:   Update  $\mathbf{w}$  using Eq. (8)
7:   if Task Ends then
8:     Update  $\theta$  using  $\mathcal{B}$  and Eq. (7)
9:     Reset transient value function,  $\mathbf{w}$ 
10:    Reset  $\mathcal{B}$ 
11:  end if
12: end for
```

CRL测试环境

Algorithm 4 PT-DQN Pseudocode (Continual Reinforcement Learning)

```
1: Initialize transient network buffer  $\mathcal{B}$ , permanent network buffer  $\mathcal{D}$ 
2: Initialize permanent network  $\theta$ , transient network  $\mathbf{w}$ 
3: Initialize target network  $\mathbf{w}_{target}$ 
4: for  $t : 0 \rightarrow \infty$  do
5:   Take action  $A_t$  according to  $\epsilon$ -greedy policy
6:   Observe  $R_{t+1}, S_{t+1}$ 
7:   Store  $(S_t, A_t, R_{t+1}, S_{t+1})$  in  $\mathcal{B}$ 
8:   Store  $(S_t, A_t, Q^{(P)}(S_t, A_t))$  in  $\mathcal{D}$ 
9:   Sample a batch of transitions  $(S_j, A_j, R_{j+1}, S_{j+1})$  from  $\mathcal{B}$ 
10:  Compute target
     $y = R_{j+1} + \gamma \max_{a' \in \mathcal{A}} (Q^{(P)}(S_{j+1}, a'; \theta) + Q^{(T)}(S_{j+1}, a'; \mathbf{w}_{target})) - Q^{(P)}(S_j, A_j; \theta)$ 
11:  Perform a gradient step on  $(y - Q^{(T)}(S_j, A_j; \mathbf{w}))^2$  with respect to  $\mathbf{w}$ 
12:  Update target network every  $N$  steps,  $\mathbf{w}_{target} = \mathbf{w}$ 
13:  if  $\text{mod}(t, k) == 0$  then
14:    Sample a batch of transitions  $(S_j, A_j, Q^{(P)}(S_j, A_j))$  from  $\mathcal{D}$ 
15:    Compute target  $\hat{y} = Q^{(P)}(S_j, A_j) + Q^{(T)}(S_j, A_j; \mathbf{w})$ 
16:    Perform a gradient step on  $(\hat{y} - Q^{(P)}(S_j, A_j; \theta))^2$  with respect to  $\theta$ 
17:    Decay transient network weights,  $\mathbf{w} = \lambda \mathbf{w}$ 
18:    Clear permanent network buffer,  $\mathcal{D} = \{\}$ 
19:  end if
20: end for
```

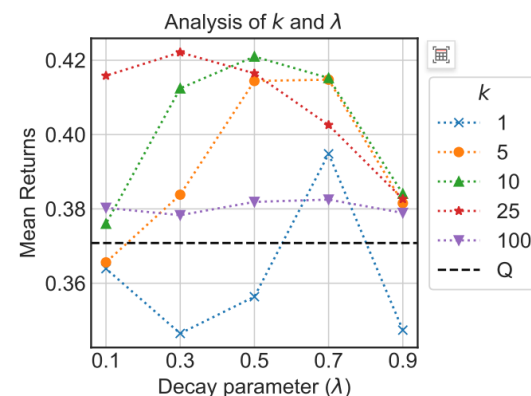


Figure 4: λ and k analysis.

(1)半持续强化学习(任务切换可以被感知): 当切换任务时瞬态网络重置重新学习; 用最新的数据去训练永久网络---**信号明确**

(2)持续强化学习(任务切换没有明显的信号): 每隔K步更新永久网络是因为检测不到任务切换, 只能周期性的更新

k与 λ 两个超参数: 对于每个 k, 都至少存在一个 λ 值, 使得所提出算法的性能超过 Q-learning 基线---**不过没有什么很明显的规律**

结果展示: Prediction and Control in Continual Reinforcement Learning

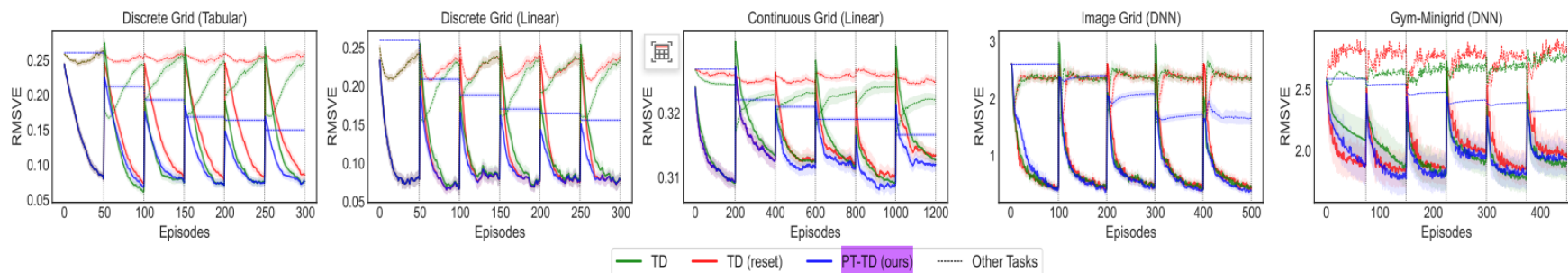


Figure 2: Prediction results on various problems. Solid lines represent RMSVE on the current task and dotted lines represent MSE on other tasks. Black dotted vertical lines indicate task boundaries.

图2: 半持续强化学习(预测任务)

①在旧任务: 误差越来越小, 有一个比较好的效果

②当前任务: 在用表格或者线性方法去充当的, 效果比较好(任务切换开始时误差小, 且误差下降的很快)

在用神经网络充当的, 效果不是很明显(任务切换开始时误差且下降速度与其他方法看上去不是很明显)

图3: 半持续强化学习(控制任务)

①任务切换开始时reward与其他方式相比没有明显的上升, 但是reward上升的速度的很快

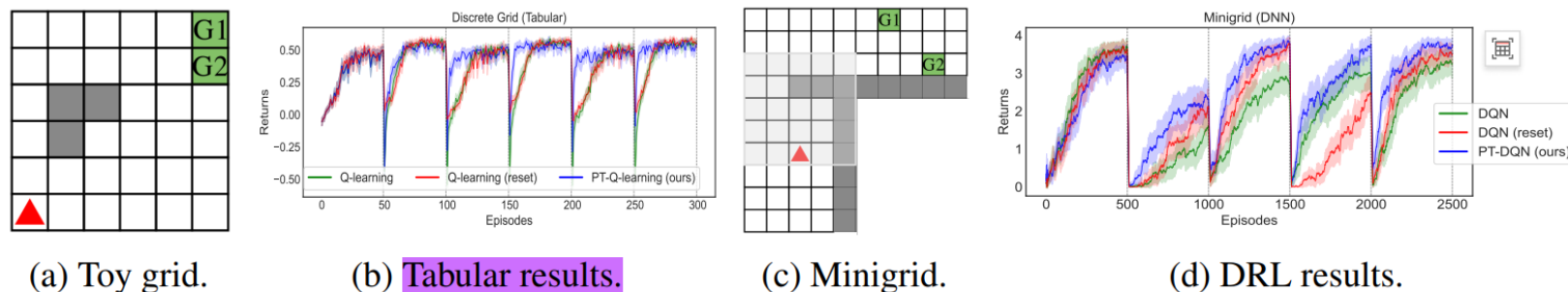


Figure 3: The environments and the corresponding results in the control setting. (b) Tabular results: episodic returns are plotted. (d) Deep RL results: returns smoothed across 10 episodes is plotted.

结果展示: Prediction and Control in Continual Reinforcement Learning

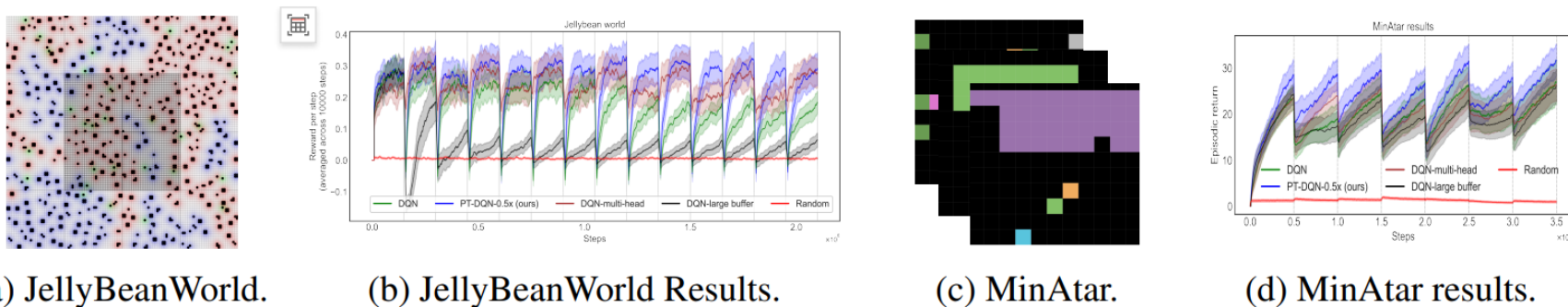


Figure 5: (a) JellyBeanWorld task. (b) Results: JellyBeanWorld. (c) MinAtar tasks. (d) Results: MinAtar.

图5: 持续强化学习

①任务切换开始时reward与其他方式相比没有明显的上升, 但是reward上升的速度很快且最终reward要高一些

现象: ①每次任务切换都会有一个明显的下降: 永久网络提供了一个基于所有任务平均值的通用基线, 但它并不包含“当前新任务”的特有细节; 切换瞬间智能体失去了针对“上一个任务”的修正能力---稳定性

②每次切换后的起点差不多: 也就是说第一个任务切换完与第二个任务切换完起点是类似的, 并没有说多学习了一个任务的永久知识就会比前一次切换起点要好。是不是可以说如果任务相似, 学一个任务就可以把所有任务的通用知识学出来? ---通用知识的作用

③每次任务切换后, 起点虽然没有一个上升的趋势, 但是RL的学习能力有一个较大的提升(快速达到一个好的效果): 任务切换后瞬态网络都要进行“旧知识清空操作”, 相当于使用了新的神经元去重新学习当前任务的知识---可塑性

总结一下

- ①**永久网络**，输出的Q值代表智能体在长期训练过程中积累的稳态知识，反映了环境的不变规律或通用策略。
- ②**瞬态网络**，输出的Q值针对当前特定任务、最新环境变化或短期上下文的动态调整量

想法：引入LLM，利用LLM的先验知识，在非平稳状态下产生一个合理的参考；同时为了降低调用LLM产生显著的计算资源消耗与延迟成本，为了实现能力与效率的平衡，在永久+瞬态双网络架构中引入了**按需触发机制**。

问题1：LLM什么时刻调用？调用LLM时LLM到底充当永久网络还是瞬态网络？

想法：**当任务切换的时候调用LLM，LLM充当瞬态网络**

理由：①当任务切换的时候，模型的效果必然会下降，此时模型还没有学到新任务的一些知识(比如奖励的改变、目标的改变等等)。这个时候需要的是切换后模型效果好一些(起点不是从零开始) + 模型快速适应新任务，这两个目的其实都是**针对当前新任务的，也就是说瞬态网络需要做的事**

②当任务切换后，调用LLM替代瞬态网络去学习新任务，快速产生当前任务的优质数据，然后去训练瞬态网络，相当于用好的数据去训练瞬态网络

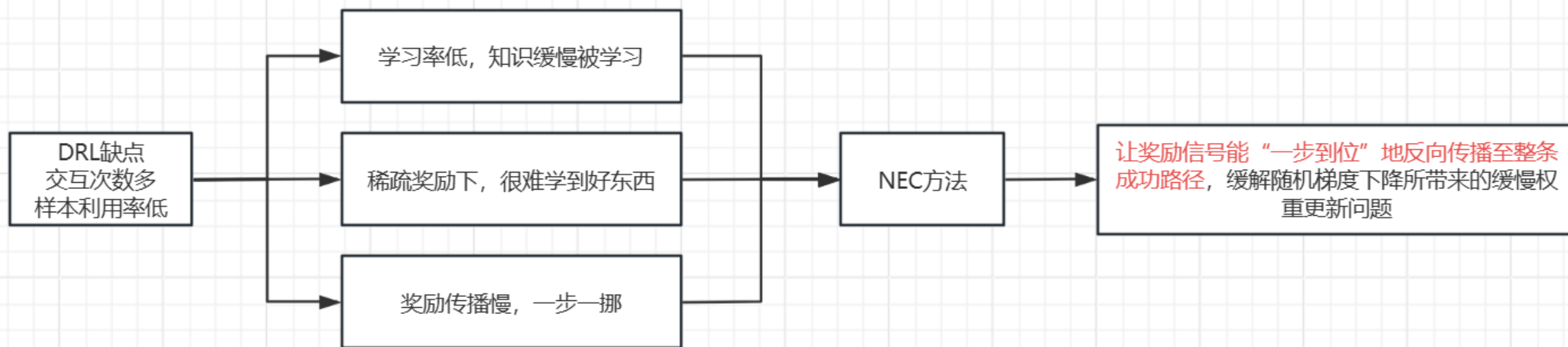
问题2：LLM能不能发挥作用？

在RL的任务中，一般输入状态S，智能体通过与环境交互确定我的任务是啥/我的目标是啥；切换任务时需要重新学习新知识(新任务改变了啥)，所以传统的CRL方法需要从头适应并且可能会遗忘旧知识

现在的问题是CRL中任务**切换是不被感知的**，尤其是切换初始时不知道什么发生了变化？这个时候调用LLM，只将状态的文本表示传给LLM，但是LLM不知道环境到底变化了什么，也是需要重新探索的？这跟传统的方法有什么区别吗？

要是将新任务的信息也传给LLM，这又比传统的神经网络相比多输入了信息；要是都输入新任务的信息又会违反CRL设定

相关研究：2017-NIPS-Neural Episodic Control



NEC由三部分组成：

- ①卷积神经网络（CNN），将原始像素转化为稳定且泛化的状态嵌入，为记忆检索提供可靠索引；
- ②动作专属的记忆模块（DND），以非参数方式即时存储和检索每个动作的历史高回报经验，实现单次试错后的快速复用；
- ③最终读取网络，通过对记忆中相似状态的加权插值，将离散的经验片段平滑转化为连续的Q值估计，从而指导智能体做出最优决策。

核心：2017-NIPS-Neural Episodic Control

记忆模块组DND:

(K (状态嵌入向量 h), V (该状态下采取动作 a 的真实回报估计))

①查找: 给定当前状态的特征向量 h , 从记忆中检索出对应的 Q 值估计

$$o = \sum_i w_i v_i, \quad (1)$$

其中 v_i 是数组 V_a 的第 i 个元素, 且

$$w_i = \frac{k(h, h_i)}{\sum_j k(h, h_j)}, \quad (2)$$

其中 h_i 是数组 K_a 的第 i 个元素, $k(x, y)$ 是向量 x 和 y 之间的核函数

- $k(h, h_i)$: 核函数 (kernel function), 衡量当前状态嵌入 h 与记忆中第 i 个状态嵌入 h_i 的“相似度”。
- 越相似 $\rightarrow k(h, h_i)$ 越大 $\rightarrow$ 权重越高。

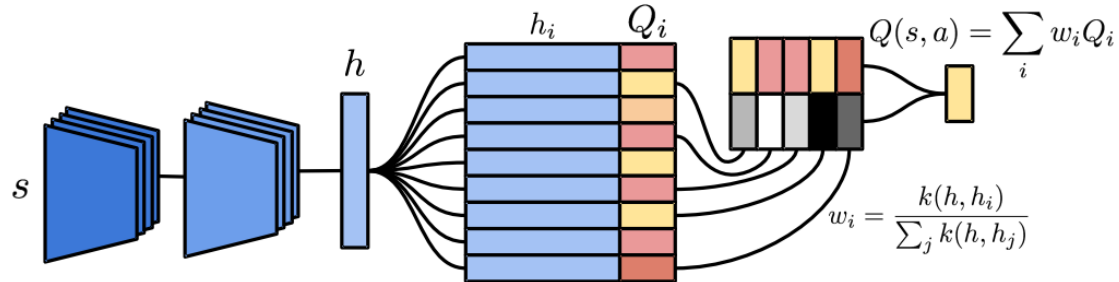
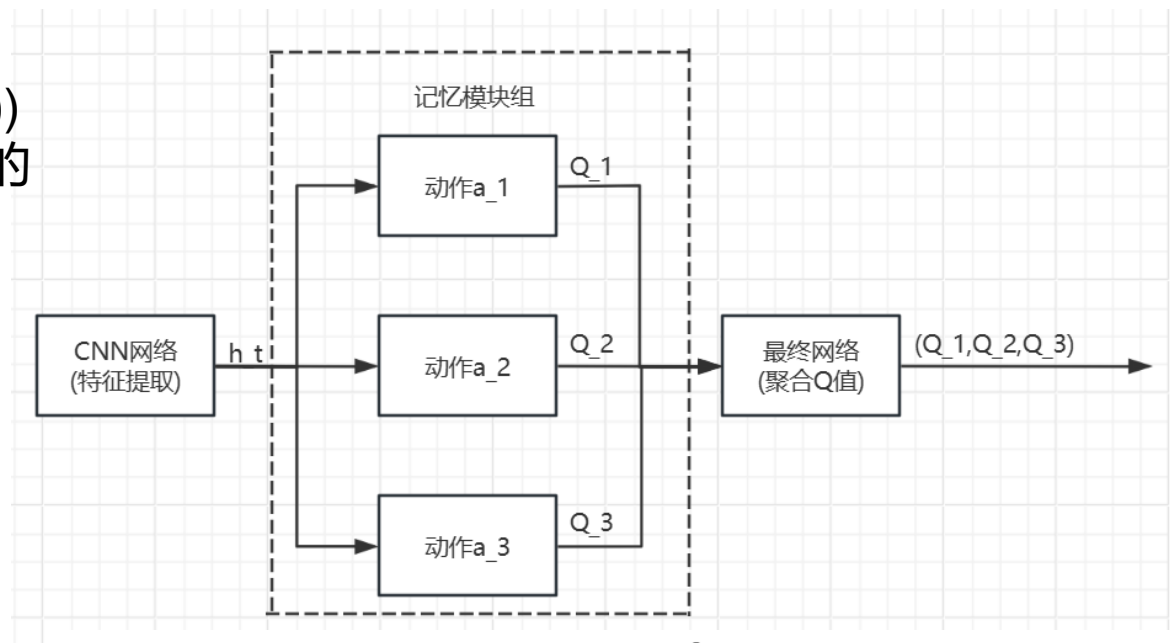
标量(o): 当前状态 h 下采取动作 a 的预期回报(图中的 Q_1, Q_2, Q_3)

v_i : 历史上在相似状态下执行动作 a 所获得的真实回报

权重(w_i): 当前状态 h 与记忆中第 i 个状态 h_i 的相似度

$$Q^{(N)}(s_t, a) = \sum_{j=0}^{N-1} \gamma^j r_{t+j} + \gamma^N \max_{a'} Q(s_{t+N}, a') \quad (3)$$

$Q_{(s_t, a)}^N$: 是通过查询所有动作对应的 DND 记忆模块得到的



②插入: 将新的 $(h, Q_{(s_t, a)}^N)$ 对写入记忆 (追加或更新)
高 α 可以一步到位接近最新目标值, 实现“顿悟式学习”

$$Q_i \leftarrow Q_i + \alpha(Q^{(N)}(s, a) - Q_i) \quad (4)$$

算法流程：2017-NIPS-Neural Episodic Control

Algorithm 1 Neural Episodic Control

$\mathcal{D}$: replay memory.

M_a : a DND for each action a .

N : horizon for N -step Q estimate.

for each episode **do**

for $t = 1, 2, \dots, T$ **do**

 Receive observation s_t from environment with embedding h .

 Estimate $Q(s_t, a)$ for each action a via (1) from M_a

$a_t \leftarrow \epsilon$ -greedy policy based on $Q(s_t, a)$

 Take action a_t , receive reward r_{t+1}

 Append $(h, Q^{(N)}(s_t, a_t))$ to M_{a_t} .

 Append $(s_t, a_t, Q^{(N)}(s_t, a_t))$ to $\mathcal{D}$.

 Train on a random minibatch from $\mathcal{D}$.

end for

end for

```
1 输入帧 s
2      ↓
3 [CNN 编码器] → h = f_θ(s)    ← 共享参数!
4      ↓
5 ┌──────────────────────────┐
6 │ 对每个动作 a: │
7 │   M_a = (K_a, V_a)        ← 独立记忆模块
8 │   o_a = Σ w_i * v_i       ← 加权平均输出
9 └──────────────────────────┘
10      ↓
11 Q(s,a) = [o_0, o_1, ..., o_{|A|-1}]
12      ↓
13 argmax_a Q(s,a) → 执行动作
```

最后回放缓冲区中存储三元组 (s_t, a_t, R_t) 训练负责特征提取的CNN网络: $\text{Loss} = (Q - Q_{(s,a)}^N)^2$

Q : 输入原始像素 s , 经过 CNN 输出对该状态下动作 a 的预测 Q 值

$Q_{(s,a)}^N$: N -step 规则计算的真正折现回报

实验结果：2017-NIPS-Neural Episodic Control

①奖励裁剪映射类游戏：

A3C、DQN 及相关算法为了训练稳定性，需要将奖励裁剪到 $[-1, 1]$ 范围内：抹平了奖励之间的相对大小

NEC 和 MFEC 不需要奖励裁剪：保留原始奖励的量级信息

Alien 涉及控制一个角色，方式1可以通过收集大量存在的物品来获取小奖励，同时避开对智能体无敌的敌人；方式2智能体可以拾取特殊道具使敌人变得脆弱，从而允许智能体攻击它们并获得远大于收集小奖励的巨额回报。

参数化方法倾向于收集小奖励，而 NEC 会主动尝试使敌人脆弱并攻击它们以获取大奖励。

②奖励不需要裁剪映射：NEC 在最大 achieved 得分方面并未超越其他算法，但它具有极高的数据效率

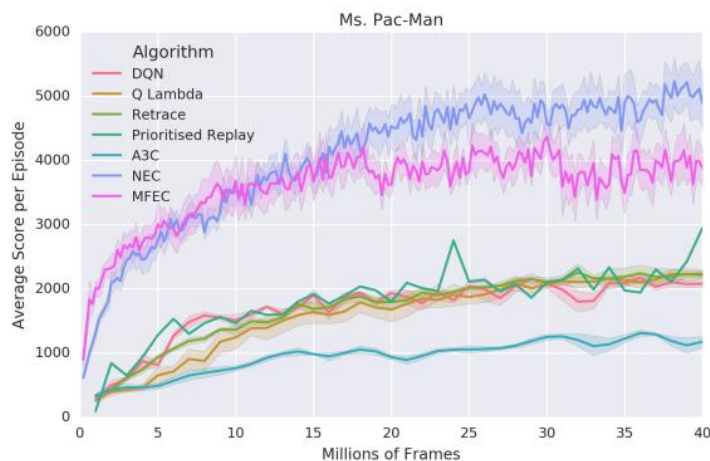


Figure 6. Learning curve on Ms. Pac-Man.

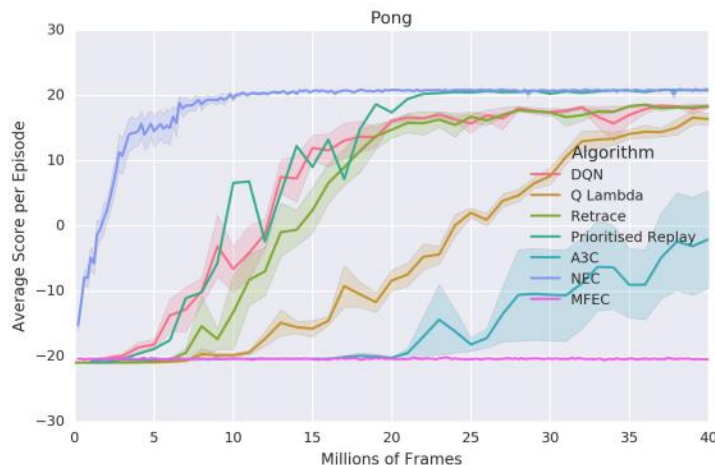
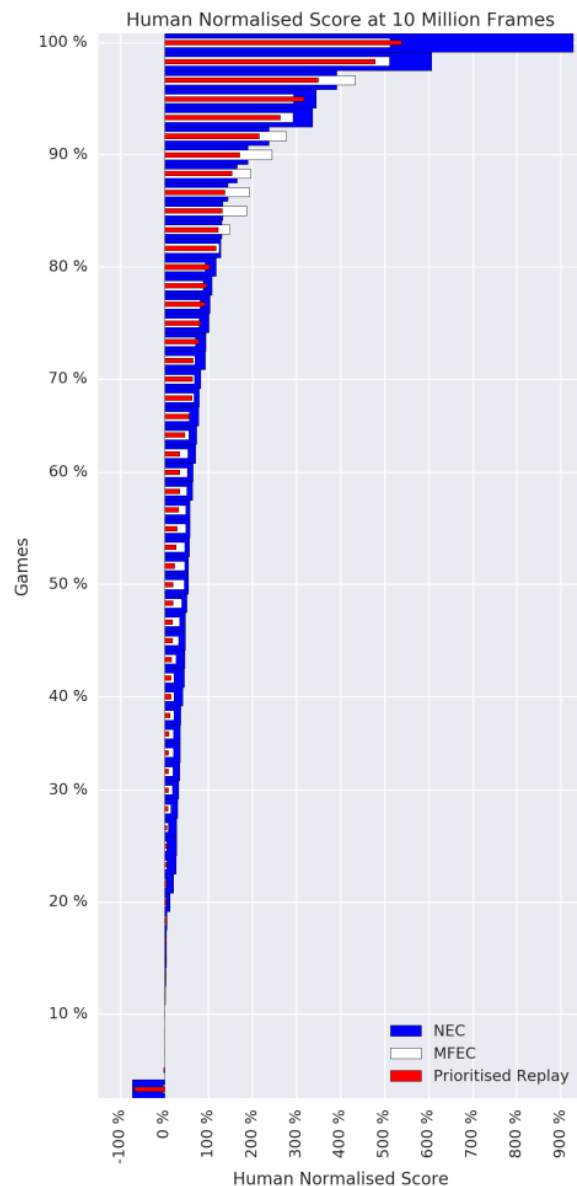
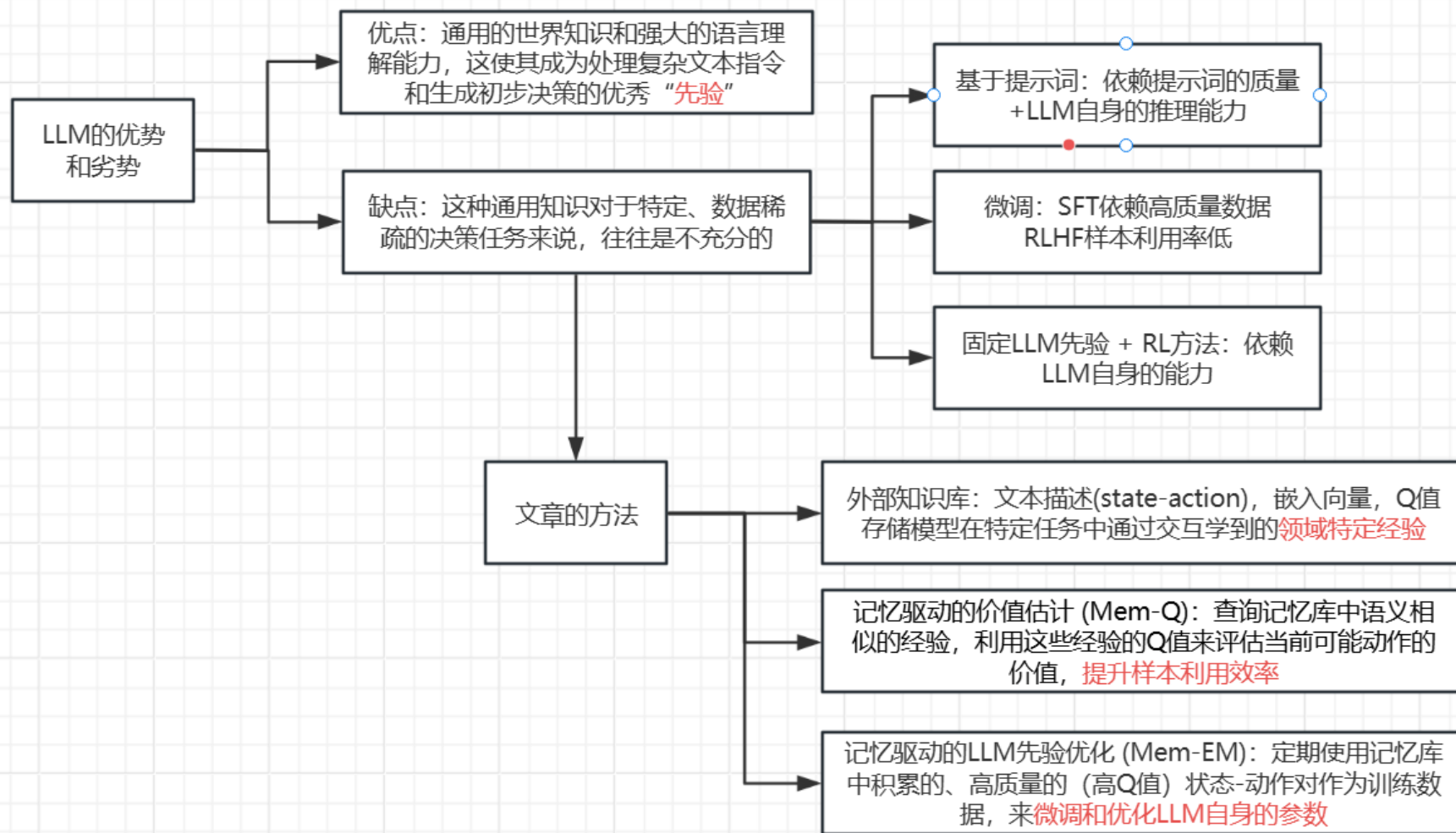


Figure 8. Learning curve on Pong.



Y 轴：游戏性能排名百分位”，
X 轴：人类标准化得分
NEC 在几乎所有排名区间都优于其他算法，尤其在高分段优势巨大，证明了其卓越的数据效率和泛化能力。

相关研究:2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models



①检索增强生成 (RAG) 中LLM具有生成语义丰富的向量化表示的能力, 利用基于LLM的嵌入来增强检索质量并改进价值估计

↓
LLM的编码器作为嵌入函数 f_{LLM} , 为每个文本形式的 (s_t, a) 生成嵌入向量 $f(s_t, a)$

②基于LLM先验的记忆驱动价值估计Mem-Q:

③记忆驱动的LLM先验优化Mem-EM

Mem-Q:2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models

Algorithm 1 Memory-Driven Q-learning (Mem-Q)

Input: Embedding function f_{LLM} , empty memory table $\mathcal{M} = \emptyset$.

Output: Updated memory table $\mathcal{M}$.

```
1: for each episode do
2:   for  $t = 1, 2, 3, \dots, T$  do
3:     Obtain all feasible actions  $\mathcal{A}_{s_t}$  for current  $s_t$ .
4:     Estimate the return  $\hat{Q}$  for each  $a \in \mathcal{A}_{s_t}$  via Eq. 3.
5:     Select action  $a_t$  using the  $\epsilon$ -greedy strategy.
6:     Execute action  $a_t$ , receive reward  $r_{t+1}$ , and observe the next state  $s_{t+1}$ .
7:   end for
8:   for  $t = T, T - 1, \dots, 1$  do
9:     Write and update memory table  $\mathcal{M}$  via Eq. 2.
10:  end for
11: end for
```

- ① 计算状态 s_t 与动作 A 组合的 Q 值: $Q(s_t, a_1), Q(s_t, a_2) \dots Q(s_t, a_n)$
- ② ϵ -greedy 选择动作并执行, 完成整个 episode
- ③ 每个 episode 结束后更新记忆表 $\mathcal{M}$

计算 Q 值的方法: LLM 的编码器作为嵌入函数 f_{LLM} , 为每个文本形式的 (s_t, a) 生成嵌入向量 $f(s_t, a)$, 找到最相近的 M 个向量, 加权得到最终的 Q 值

$\hat{Q}$, which is defined as

$$\hat{Q}(s_t, a) = \sum_{i \in \mathcal{N}_M(f(s_t, a))} w_i Q(s^{(i)}, a^{(i)}), \quad w_i = \frac{k(h, h_i)}{\sum_{j \in \mathcal{N}_M} k(h, h_j)}, \quad h = f(s_t, a), h_i = f(s^{(i)}, a^{(i)}), \quad (3)$$

记忆表更新:

记忆中没有的直接存入(将回报 R 设置为 Q),
记忆表中有的取原有 Q 值和当前回报 R_t 的最大

$$Q(s_t, a_t) \leftarrow \begin{cases} R_t, & \text{if } (s_t, a_t) \notin \mathcal{M}, \\ \max\{Q(s_t, a_t), R_t\}, & \text{otherwise,} \end{cases} \quad (2)$$

where $R_t = \sum_{i \geq t} \gamma^{i-t} r_i$ represents a Monte Carlo estimate of the cumulative discounted reward (i.e., the return-to-go).

Mem-Q:2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models

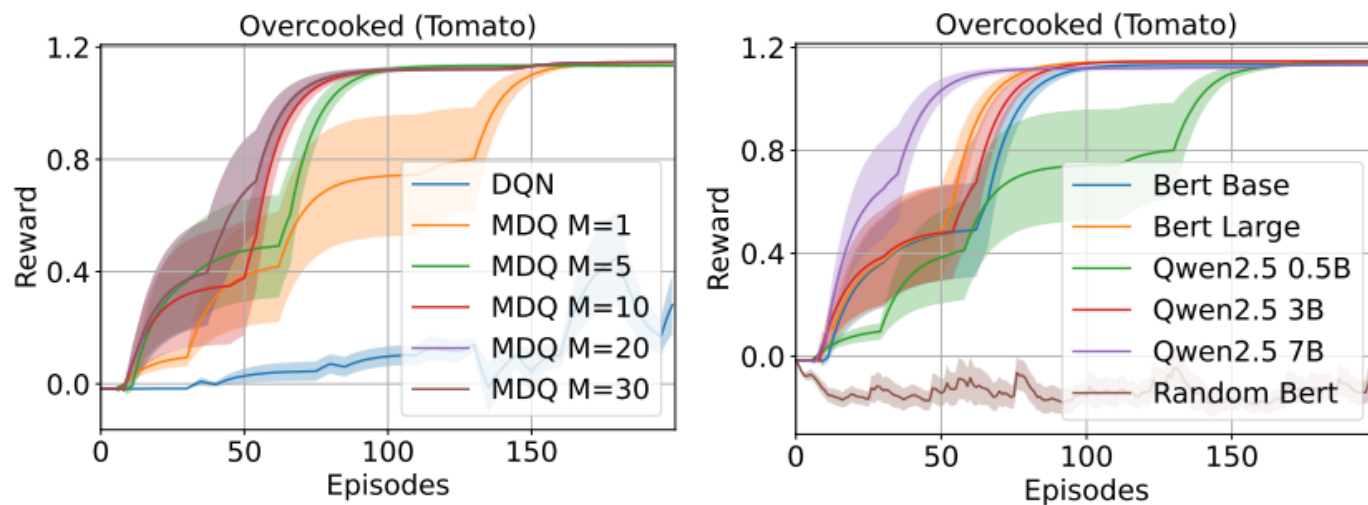


Figure 2: Results of memory-drive Q-learning on Overcooked. Left: effect of the number of retrieved (s, a) pairs for value estimation; Right: effect of different LLMs on representations.

- ①检索邻居数量 M 的影响: M 越多, Mem-Q的性能显著提升, 学习速度明显加快, 最终达到的最高奖励也更高
 - ②模型能力的影响: 随着LLM的规模(参数量)增大, Mem-Q的性能持续提升
- Random Bert 的表现非常差, 表明了高质量的、有意义的语义表示是成功的关键

Mem-Q:2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models

Algorithm 2 Memory-Driven Expectation Maximization (Mem-EM) with LLM Prior Refinement

Input: LLM action prior p_{LLM_θ} , memory table $\mathcal{M} = \emptyset$

Output: Refined LLM prior p_{LLM_θ} and memory table $\mathcal{M}$

```
1: for episode  $i = 1$  to  $N$  do
2:   for step  $t = 1, 2, \dots, T$  do
3:     Sample action candidates from LLM prior  $\mathcal{C}^K(s_t) \sim p_{LLM_\theta}(\cdot|s_t)$ .
4:     Estimate Q-values  $\hat{Q}(s_t, a)$  for  $a \in \mathcal{C}^K(s_t)$  via Eq. 3.
5:     Select action  $a \sim \text{Multinomial}(\exp(\hat{Q}(s, a_1)/\tau) / \sum_k \exp(\hat{Q}(s, a_k)/\tau))$ .
6:     Execute  $a_t$ , observe reward  $r_{t+1}$  and next state  $s_{t+1}$ 
7:   end for
8:   for step  $t = T$  down to 1 do
9:     Write and update memory table  $\mathcal{M}$  via Eq. 2.
10:  end for
11:  if  $i$  reaches update interval then
12:    Refine LLM prior  $p_{LLM_\theta}$  using stored  $(s, a)$  pairs from  $\mathcal{M}$  via Eq. 7.
13:  end if
14: end for
```

Note: Orange text highlights LLM-guided exploration steps that differ from the Mem-Q in Algorithm 1; green text indicates prior refinement operations.

①状态 s_t 的动作由LLM生成然后计算Q值:

$Q(s_t, a_1), Q(s_t, a_2) \dots Q(s_t, a_n)$

②利用softmax输出动作, Q值越高的动作

③达到固定的间隔, 更新一次LLM参数

◆ 公式 (7): 基于记忆表的记忆加权损失

$$\theta_{k+1} = \arg \min_{\theta} \sum_{(s,a) \sim \mathcal{M}} \left[\frac{\exp(Q(s,a)/\tau)}{\sum_{(s',a') \sim \mathcal{M}} \exp(Q(s',a')/\tau)} \log p_{LLM_\theta}(a|s) \right]$$

或更简洁地写作:

$$\theta_{k+1} = \arg \min_{\theta} \sum_{(s,a) \sim \mathcal{M}} w(s,a) \cdot \log p_{LLM_\theta}(a|s) \quad \text{其中 } w(s,a) = \frac{\exp(Q(s,a)/\tau)}{\sum_{(s',a') \sim \mathcal{M}} \exp(Q(s',a')/\tau)}$$

◆ 解读

这是实际实现中使用的监督微调目标:

- 从记忆表 $\mathcal{M}$ 中采样经验。
- 每个样本 (s, a) 被赋予一个权重 $w(s, a) \propto \exp(Q(s, a)/\tau)$ 。
- 目标是最小化加权负对数似然 (即最大化加权对数似然)。

✅ **本质:** 用记忆中已知的高 Q 值经验作为 “软标签”, 并根据其 Q 值进行重要性采样, 来训练 LLM。

$\log p_{LLM_\theta}(a|s)$: LLM 在给定状态 s 下生成动作 a 的对数概率
目标是让这个对数概率在高 Q 值的动作上尽可能大

Mem-Q:2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models

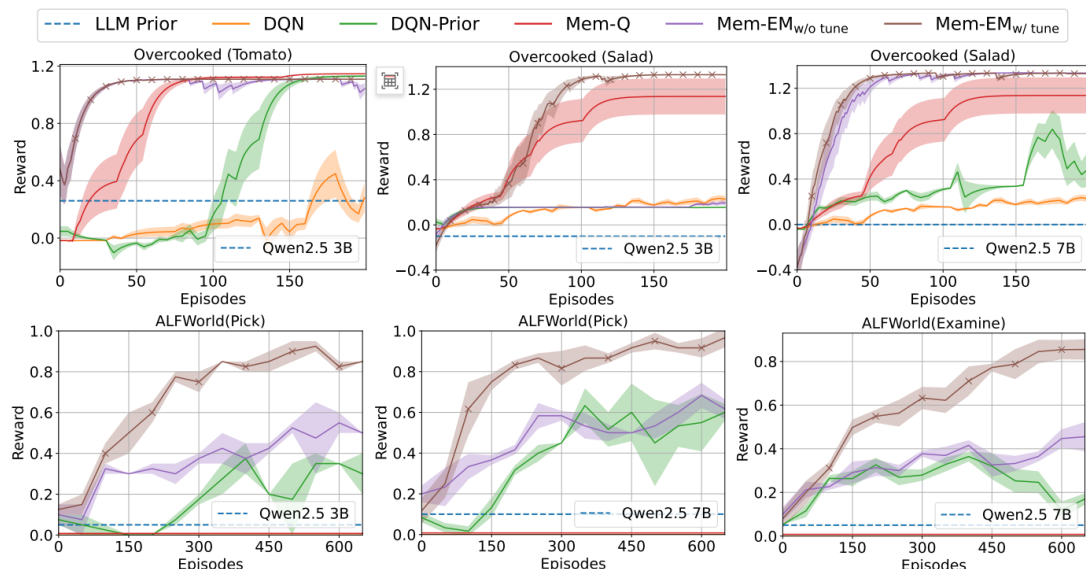


Figure 3: Results of comparison with baselines. We plot the mean and standard error of the cumulative reward. The dashed line represents directly prompting the LLM prior to generating actions given state information, with the corresponding LLM version specified. The 'x' markers for Mem-EM_{w/ tune} indicate the time steps when the LLM prior is fine-tuned.

- ①Mem-EM_{w/ tune} 是最强的，它在所有基线中表现最好
- ②利用LLM先验来生成有价值的行动候选，优于探索完整原始动作空间的模型：DQN-Prior 和 Mem-EM_{w/o tune} 优于 DQN 和 Mem-Q
- ③Mem-EM_{w/ tune} 显著优于 Mem-EM_{w/o tune}：通过从记忆库中提取高质量的经验（高回报的状态-动作对），不断更新LLM学习特定任务的知识

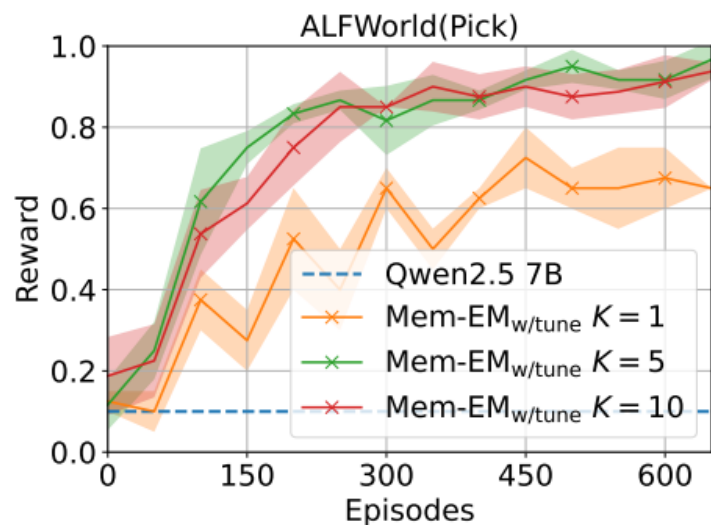
Table 1: Results on the generalization ability of ALFWorld(Pick). All trainable models are trained with $K = 5$ action candidates, and their performance is evaluated using different values of K .

Baseline	Unseen Tasks				Seen Tasks			
	K=5	K=10	K=15	K=20	K=5	K=10	K=15	K=20
LLM	0.19				0.1			
LLM + Q _{DQN-Prior}	0.16	0.19	0.14	0.22	0.6	0.70	0.70	0.65
LLM + Q _{Mem-EM_{w/tune}}	0.22	0.27	0.35	0.22	0.65	0.80	0.85	0.65
LLM _{Mem-EM_{w/tune}}	0.59				0.85			
LLM _{Mem-EM_{w/tune}} + Q _{DQN-Prior}	0.59	0.54	0.46	0.46	0.90	0.85	0.90	0.75
LLM _{Mem-EM_{w/tune}} + Q _{Mem-EM_{w/tune}}	0.81	0.65	0.62	0.68	0.95	0.95	1.00	0.95

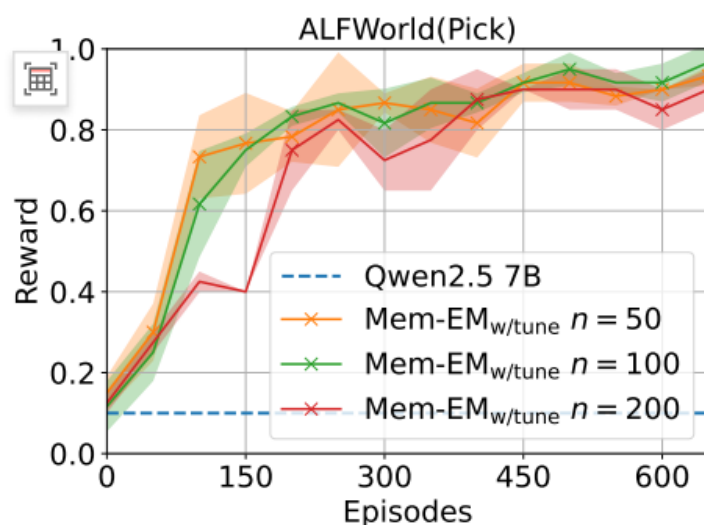
(1)已见任务上效果都很好

(2)未见任务上:①单纯的优化Q值估计方式，效果并不明显---Q值的估计是基于历史经验的，无法提供解决新任务所需的领域知识或逻辑推理
②优化LLM本身可以显著提升效果---从记忆库中提取高质量的数据

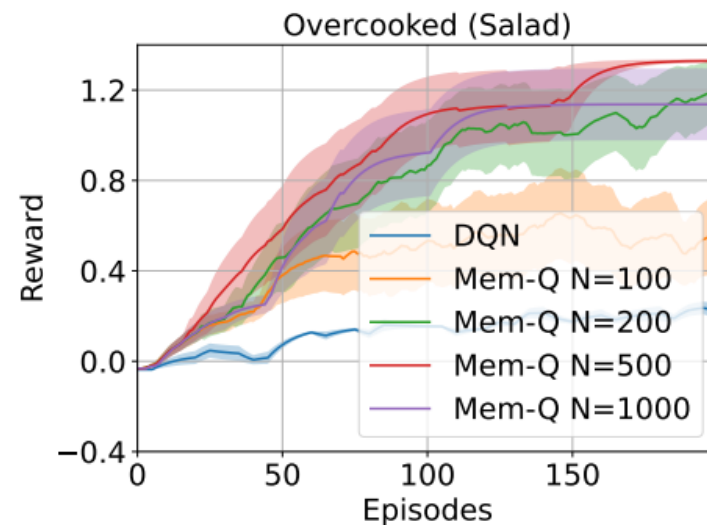
Mem-Q:2025-Memory-Driven Self-Improvement For Decision Making With Large Language Models



(a) Action candidates



(b) Tuning interval



(c) Memory capacity

- ①筛选动作数量：通过结合多个候选动作和Q值优化选择，实现了比传统Actor-Critic($K=1$)更高的样本效率，也就是说可以更有效地利用经验
- ②LLM微调频率：性能对更新间隔不敏感，即使更新间隔较长，也能保持很强的性能
- ③记忆表容量：不需要很大的内存，只要它包含了那些关键的、高价值的经验即可实现好的效果；记忆库采用了LRU替换策略